How Java is platform Independent ?

Test.java (Source Code)
⇩
Java Compiler (javac)
⇩
Test.class (byte code)
⇩
JVM (Java Virtual Machine)
⇩
With the support of O.S output will be generated.

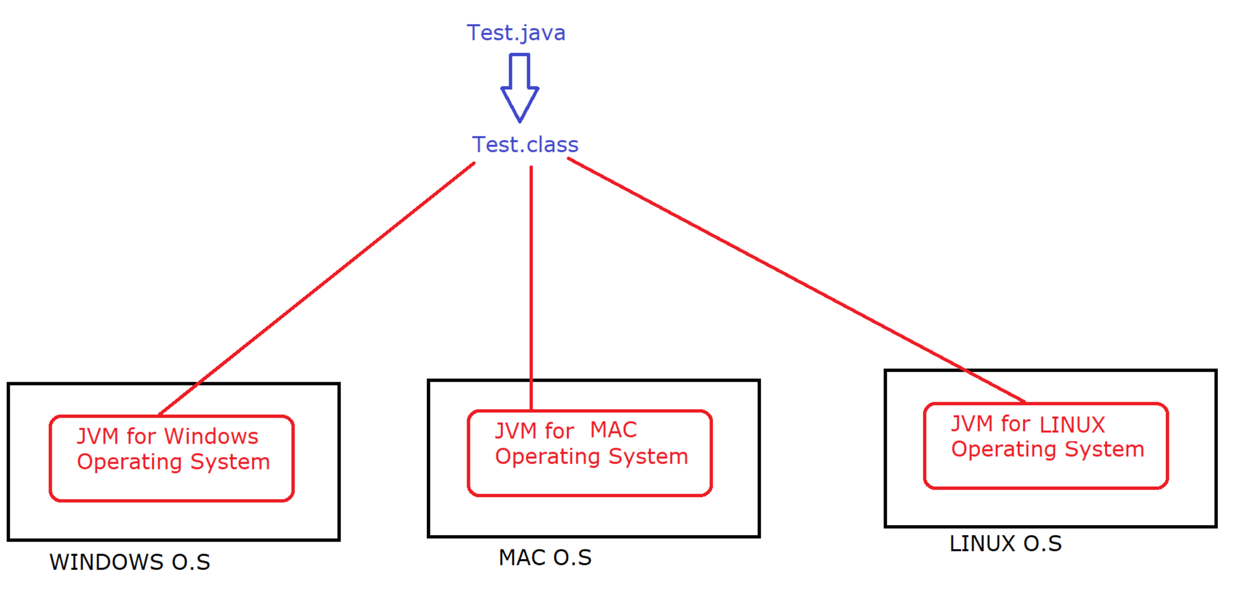
Role of Java Compiler :
----
1) Syntax verification

2) Compatibility issues L.H.S = R.H.S

3) It will convert Java Source code into **byte code.**

Java is a platform independent language. Whenever we write a java program, the extension of java program must be **.java.**

Now this .java file we submit to java compiler (javac) for **compilation process**. After successful compilation the compiler will generate a very **special byte code file** i.e. .class file (also known as bytecode). Now this .class file we submit to JVM for execution purpose.



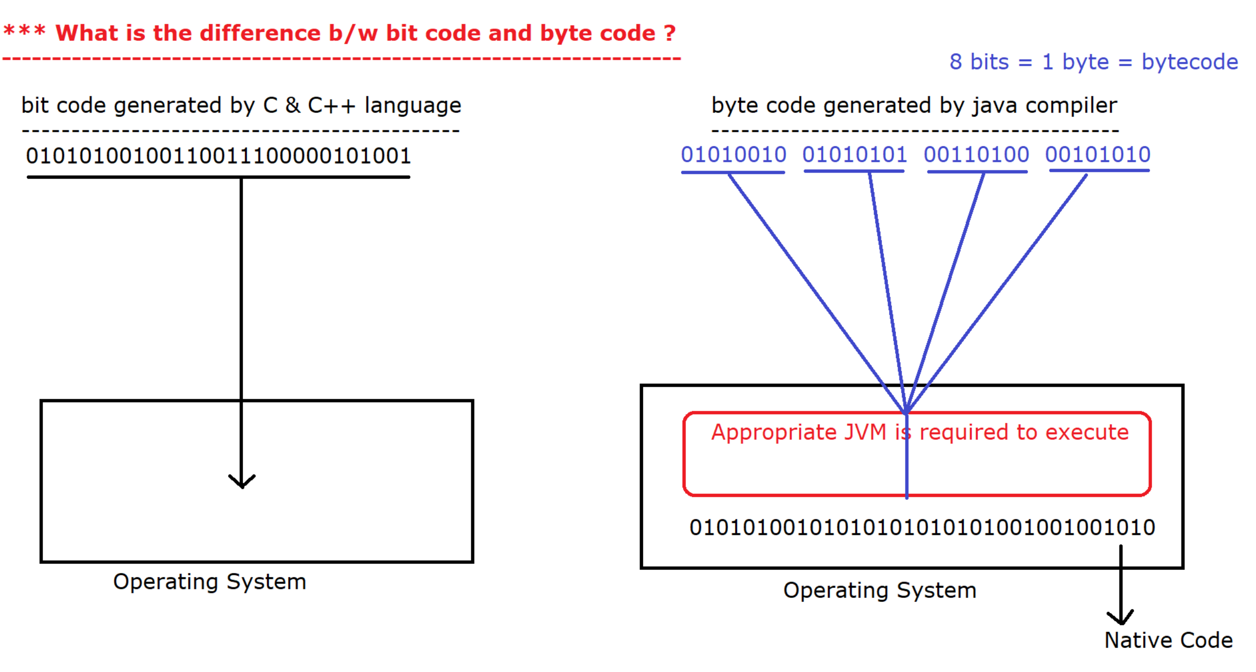
The role of JVM is to **load and execute** the .class file. Here JVM plays a major role because it converts the .class file into **appropriate machine code instruction** (Operating System format) so java becomes platform independent language and it is highly suitable for website development.

Note: - Each OS (Windows, Linux, Mac) has its own JVM implementation that means JVM is **platform dependent** technology whereas Java is platform independent technology.

JVM internally contains **an interpreter** so it executes the code line by line. It is written in 'C' language hence platform dependent.

Note: All the browsers internally contain JVM are known as JEB (Java Enabled Browsers) browser.

***** What is the difference b/w bit code and byte code ?**



Bit code is directly understood by Operating System but on the other hand byte code is understood by JVM, JVM is going to convert this byte code into appropriate machine understandable format.

What is the difference between Compiler(Javac) and Interpreter (JVM contains an Interpreter)

Compiler	Interpreter
1) It scans the entrire program once.	1) It scans line by line.
2) It will display all the errors and warning at a time.	2) It will display the runtime error only once because it executes line by line.
3) Once compilation is successful, It will generate a .class file so, separate memory is required.	3) No separate memory is required because execution is done line by line at the same time.
4) After .class file generation, execution is fast.	4) Due to line by line execution, Execution is slow.
5) We can delete .java file after successful compilation.	5) We cannot delete .class file becuase interpreter will accept one line and execute it concurrently.

***** What is the difference b/w JDK, JRE, JVM and JIT compiler ?**
